Computer Vision Homework 5

Name: NICKHOLAS

Student ID: M11515801

Department: NTUST CSIE

Write a program to do the following requirement.

I am using python 3.10 to implement all the gray-scale morphology requirements.

Using OpenCV (cv2) to read and save images, and the processing is done on NumPy arrays.

Below are the settings used for every part before jumping into each requirement:

(a) lena.bmp is read directly as a gray-scale image (values 0 to 255). No binarization is used, and all morphology operations work on the gray values.

```python
gray = cv2.imread("lena.bmp", cv2.IMREAD_GRAYSCALE)
if gray is None:
    print("Cannot read lena.bmp")
    exit()
```

(b) Using octagonal 3-5-5-5-3 kernel. With this flat kernel, dilation takes the local maximum, and erosion takes the local minimum of the gray values inside the kernel range.

```python
# Octagonal 3-5-5-5-3 kernel, origin at the center
OCTAGON = [(dy, dx)
           for dy, half in [(-2, 1), (-1, 2), (0, 2), (1, 2), (2, 1)]
           for dx in range(-half, half + 1)]
```

(c) A helper function shift is used in all four operations. Pixels that fall outside the image are filled with 0 for dilation and 255 for erosion, so they never affect the local maximum or minimum.

```python
def shift(img, dy, dx, fill):
    h, w = img.shape
    out = np.full_like(img, fill)
    ys, ye = max(0, -dy), min(h, h - dy)
    xs, xe = max(0, -dx), min(w, w - dx)
    out[ys:ye, xs:xe] = img[ys + dy:ye + dy, xs + dx:xe + dx]
    return out
```

Below are the results and explanations

(a) Dilation



Explanation:

```python
# (a) Dilation
def dilation(img, kernel):
    out = np.zeros_like(img)
    for dy, dx in kernel:
        out = np.maximum(out, shift(img, -dy, -dx, 0))
    return out
```

The dilation function is defined. For each pixel, the maximum gray value within the 3-5-5-5-3 kernel range is assigned to the center pixel, so the bright regions expand and small dark details shrink. The overall image becomes brighter.

(b) Erosion



Explanation:

```python
# (b) Erosion
def erosion(img, kernel):
    out = np.full_like(img, 255)
    for dy, dx in kernel:
        out = np.minimum(out, shift(img, dy, dx, 255))
    return out
```

For the erosion function, the minimum gray value within the 3-5-5-5-3 kernel range is assigned to the center pixel. As a result, the bright regions shrink, thin bright lines and small bright spots are removed, and the overall image becomes darker.

(c) Opening



Explanation:

```python
# (c) Opening
def opening(img, kernel):
    return dilation(erosion(img, kernel), kernel)
```

Opening runs erosion first, then dilation with the same kernel. The erosion removes small bright spots and thin bright lines, then the dilation grows the remaining bright regions back to roughly their original size and brightness. The removed parts cannot come back, so the result keeps the large bright shapes without the small bright details.

(d) Closing



Explanation:

```python
# (d) Closing
def closing(img, kernel):
    return erosion(dilation(img, kernel), kernel)
```

Closing runs dilation first, then erosion with the same kernel. The dilation fills small dark holes and narrow dark gaps, then the erosion shrinks the bright regions back to roughly their original size. The filled parts stay filled, so the result keeps the overall shape of the bright regions without the small dark details.